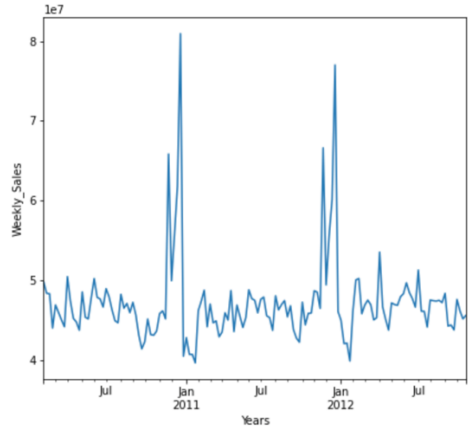
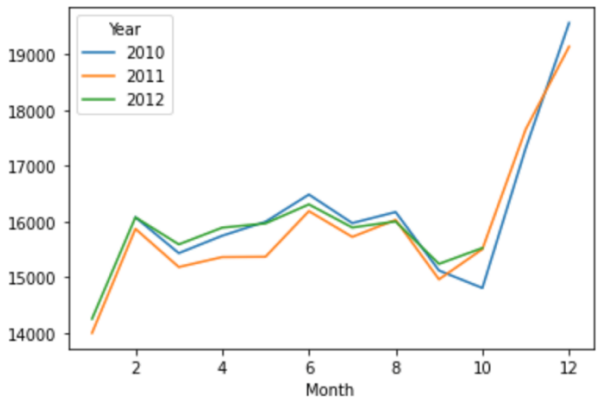
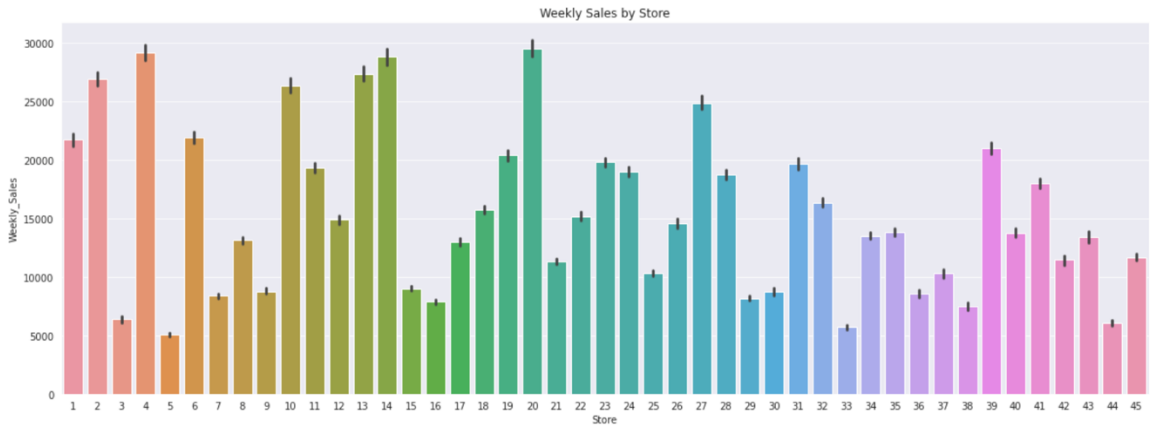
Walmart Sales Forecasting Exercise

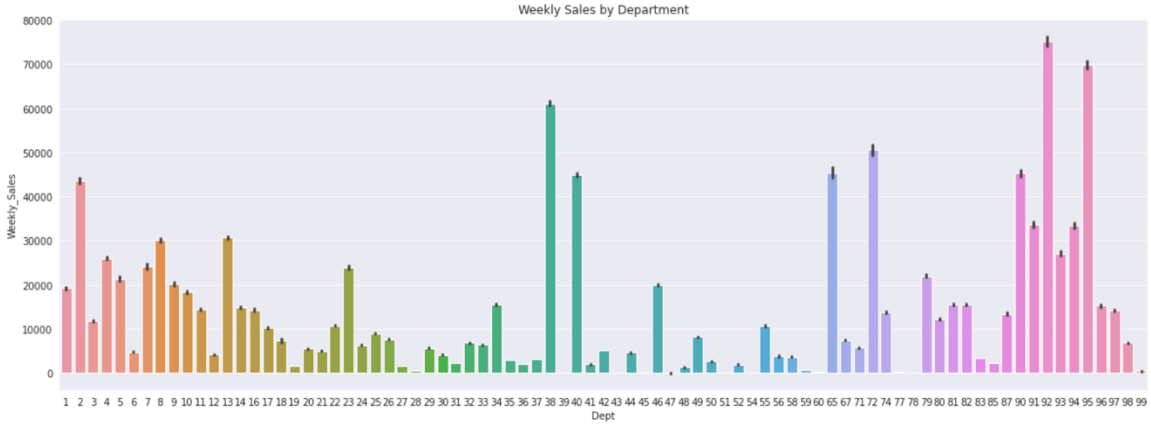
Data explorations:

This training data consists of sales data for 45 Walmart stores from 2010/2/5 to 2012/10/26. Based on the structure of the data, it is feasible to tackle this sales forecasting problem from a time series perspective. When looking at time series data, the main patterns we consider are repeating patterns, static patterns, and trends. In order to see whether there are repeating or seasonal patterns, I first plotted the annual weekly sales data. We can see a seasonal effect with a cycle of 12 months. November and December are the peak months for weekly sales. Then looking at the distribution of aggregated sales data for each store and each department, there is a lot of heterogeneity in the distribution.



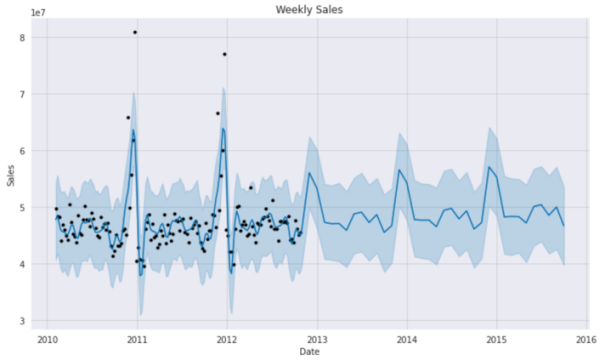


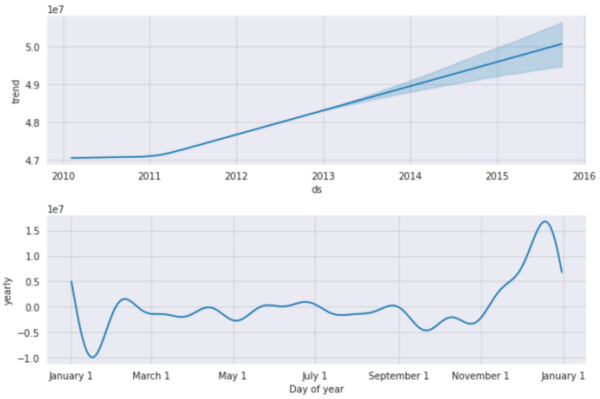




Model fitting:

I decided to use FBProphet to forecast the sales because 1. Prophet is a procedure for forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects. 2. It works best with time series that have strong seasonal effects and several seasons of historical data. In our data, the seasonal effect is clear. 3. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well. Prophet can automatically detect changes in trends by selecting changepoints from the data. The prediction is made for overall Walmart sales across all stores. After making the predictions, I extracted the trend and weekly sales by day of year. It shows an increasing trend for overall weekly sales in Walmart. Regarding day of the year, the worst sale happens in January and February while the best sale happens from November to December, which confirms the pattern we detected from the previous plot.





Limitations:
Simply forecasting sales based on historical sales data by utilizing a time series forecasting model can overlook some factors that may also be crucial to store sales. Other features such as store size, markdown, holidays, etc. can play a big role in the forecasting model as well. If time is allowed, it would be better to try out different models and compare the results. Additionally, FBProphet is an advanced level of forecasting tool which contains various mechanism in the process. Without knowing fully how the mechanism works can cause potential risks. In future analysis, it would be better to separate the forecasting of sales based on store and department differences since there are a lot of heterogeneity. Additional correlation plots are also necessary to decide the factors to consider in order to build a store-wise forecasting model.

```
import pandas as pd
from google.colab import files
import io
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving features.csv to features.csv
Saving sampleSubmission.csv to sampleSubmission.csv
Saving stores.csv to stores.csv
Saving test.csv to test.csv
Saving train.csv to train.csv
```

```
train = pd.read_csv(io.BytesIO(uploaded['train.csv']))
test = pd.read_csv(io.BytesIO(uploaded['test.csv']))
features = pd.read_csv(io.BytesIO(uploaded['features.csv']))
# stores = pd.read_csv(io.BytesIO(uploaded['stores.csv']))
# sampleSubmission = pd.read_csv(io.BytesIO(uploaded['sampleSubmission.csv']))
```

```
features.tail()
# print(train.head())
```

	Store	Date	Temperature	Fuel_Price	MarkDown1	MarkDown2	MarkDown3	Ma
8185	45	2013-06-28	76.05	3.639	4842.29	975.03	3.00	
8186	45	2013-07-05	77.50	3.614	9090.48	2268.58	582.74	
8187	45	2013-07-12	79.37	3.614	3789.94	1827.31	85.72	
8188	45	2013-07-19	82.84	3.737	2961.49	1047.07	204.19	
8189	45	2013-07-26	76.06	3.804	212.02	851.73	2.06	

```
# changing datatype to datetime format
train['Date']=pd.to_datetime(train['Date'])
test['Date']=pd.to_datetime(test['Date'])

# convert date data into day of week/month/year
train['Month']=train['Date'].dt.month
train['Year']=train['Date'].dt.year
train['Dayofweek']=train['Date'].dt.dayofweek

test['Month']=test['Date'].dt.month
test['Year']=test['Date'].dt.year
```

```
test['Dayofweek']=test['Date'].dt.dayofweek
```

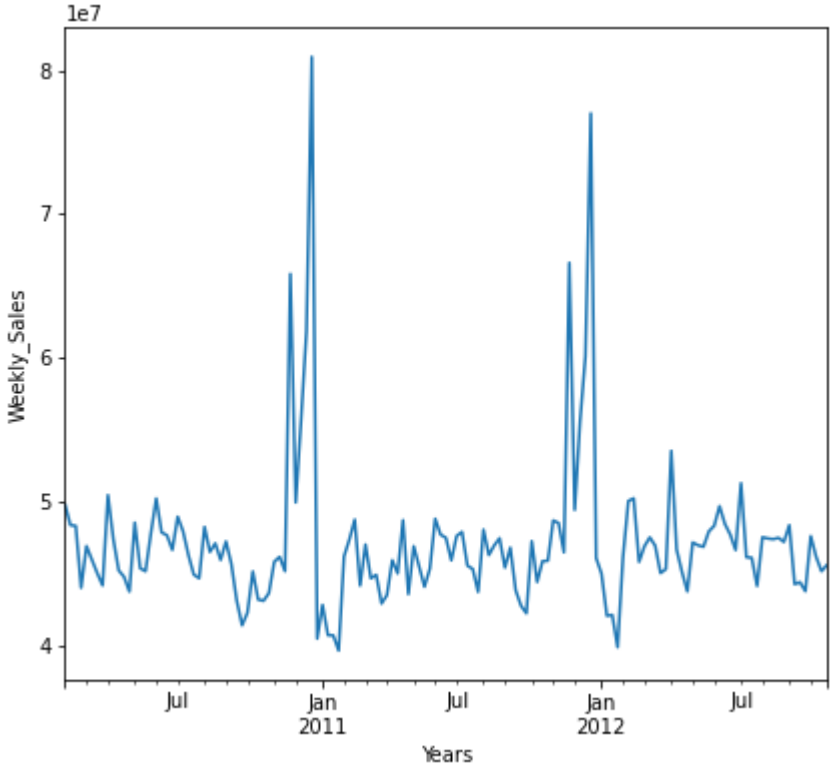
```
train.head()
```

	Store	Dept	Date	Weekly_Sales	IsHoliday	Month	Year	Dayofweek
0	1	1	2010-02-05	24924.50	False	2	2010	4
1	1	1	2010-02-12	46039.49	True	2	2010	4
2	1	1	2010-02-19	41595.55	False	2	2010	4
3	1	1	2010-02-26	19403.54	False	2	2010	4
4	1	1	2010-03-05	21827.90	False	3	2010	4

```
# set the dates as the index of the dataframe (treat it as time-series dataframe)
train.set_index('Date',inplace=True)
test.set_index('Date',inplace=True)
```

```
#plot the total sales for all stores
sales=train.groupby('Date')['Weekly_Sales'].sum()

fig = plt.figure(figsize=(15, 6))
ax1 = fig.add_subplot(121)
sales.plot(ax=ax1)
plt.xlabel('Years')
plt.ylabel('Weekly_Sales')

# from statsmodels.graphics.tsaplots import plot_acf
# ax2 = fig.add_subplot(122)
# plot_acf(sales,ax=ax2)
# plt.title("Autocorrelation", fontsize=15)
# plt.xlabel("Lag",fontsize=15)
# plt.ylabel("Correlation", fontsize=15)
# plt.show()

monthly_sales = pd.pivot_table(train, values = "Weekly_Sales", columns = "Year", ir
monthly_sales = monthly_sales.reindex(index = [1,2,3,4,5,6,7,8,9,10,11,12])
monthly_sales.plot()
```

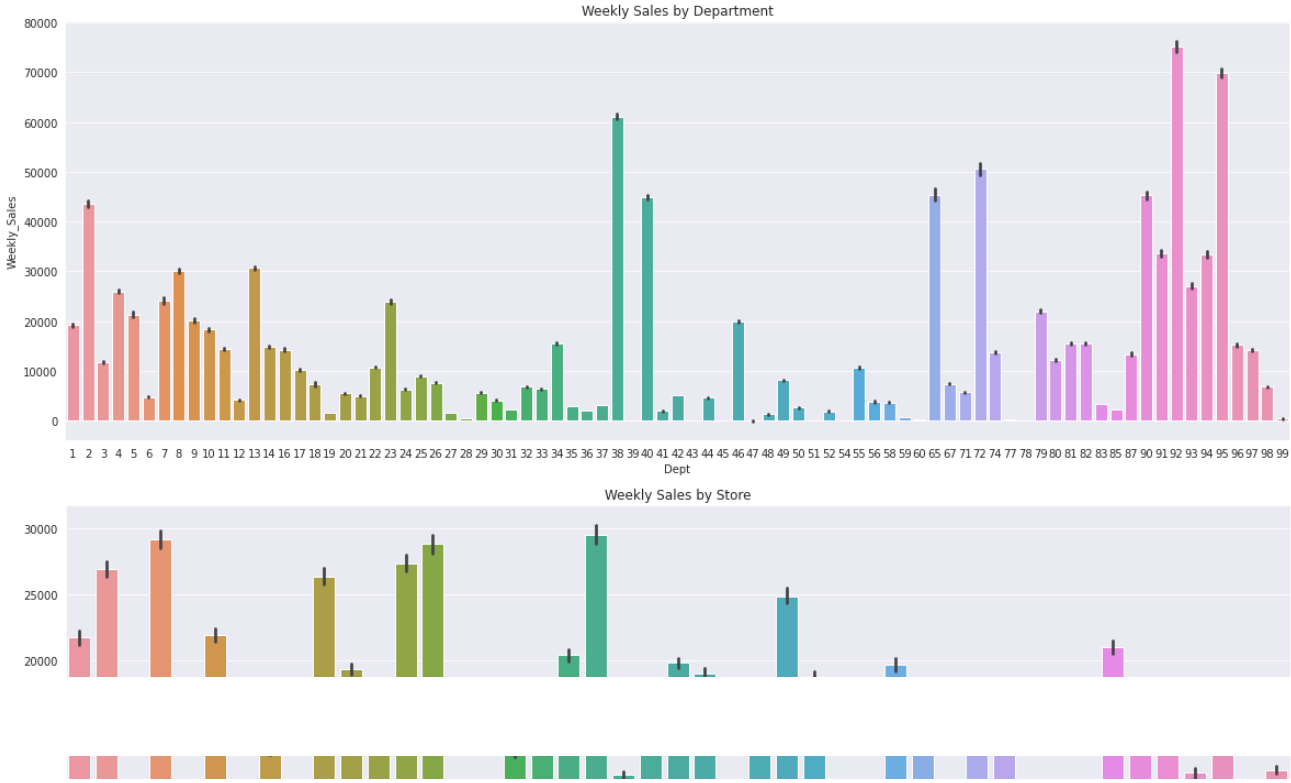
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ff3b6407a50>
```



Repeating patterns: We can see a seasonal effect with a cycle of 12 months. November and December are the peak months for weekly sales.

```
import seaborn as sns
plt.figure (figsize = (20,7))
sns.set_style('darkgrid')
sns.barplot(data = train, x = 'Dept', y =  'Weekly_Sales').set_title('Weekly Sales

plt.figure (figsize = (20,7))
sns.set_style('darkgrid')
sns.barplot(data = train, x = 'Store', y =  'Weekly_Sales').set_title('Weekly Sales
```

```
Text(0.5, 1.0, 'Weekly Sales by Store')
```



Forecasting future sales using fbprophet

```
from fbprophet import Prophet
```

```
train=train.groupby(['Date'])[['Weekly_Sales']].sum()
train['ds'] = train.index
train = train[['ds', 'Weekly_Sales']].rename(columns={'ds': 'ds', 'Weekly_Sales':
```

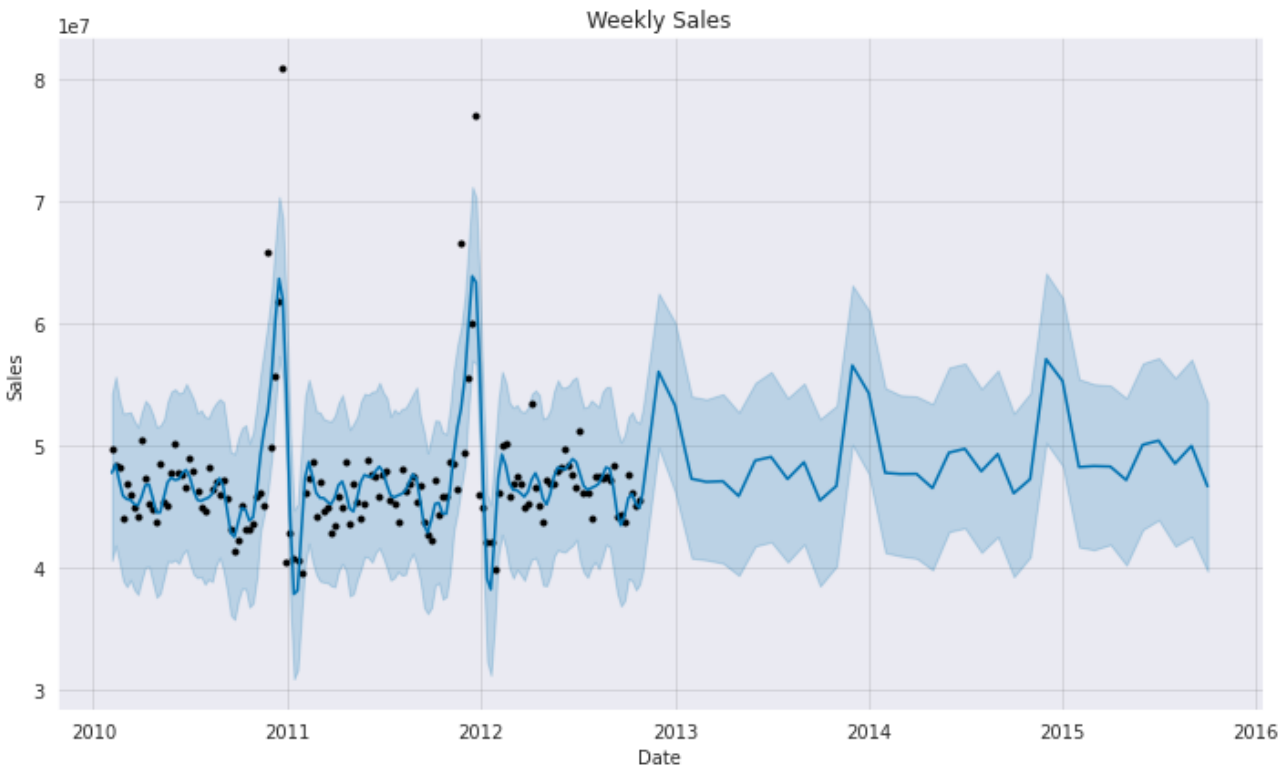
```
sales_model = Prophet(seasonality_mode='additive',interval_width=0.95)
sales_model.fit(train)
sales_forecast = sales_model.make_future_dataframe(periods=36, freq='MS')
sales_forecast = sales_model.predict(sales_forecast)
```

```
INFO:fbprophet:Disabling weekly seasonality. Run prophet with weekly_seasonal
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
```

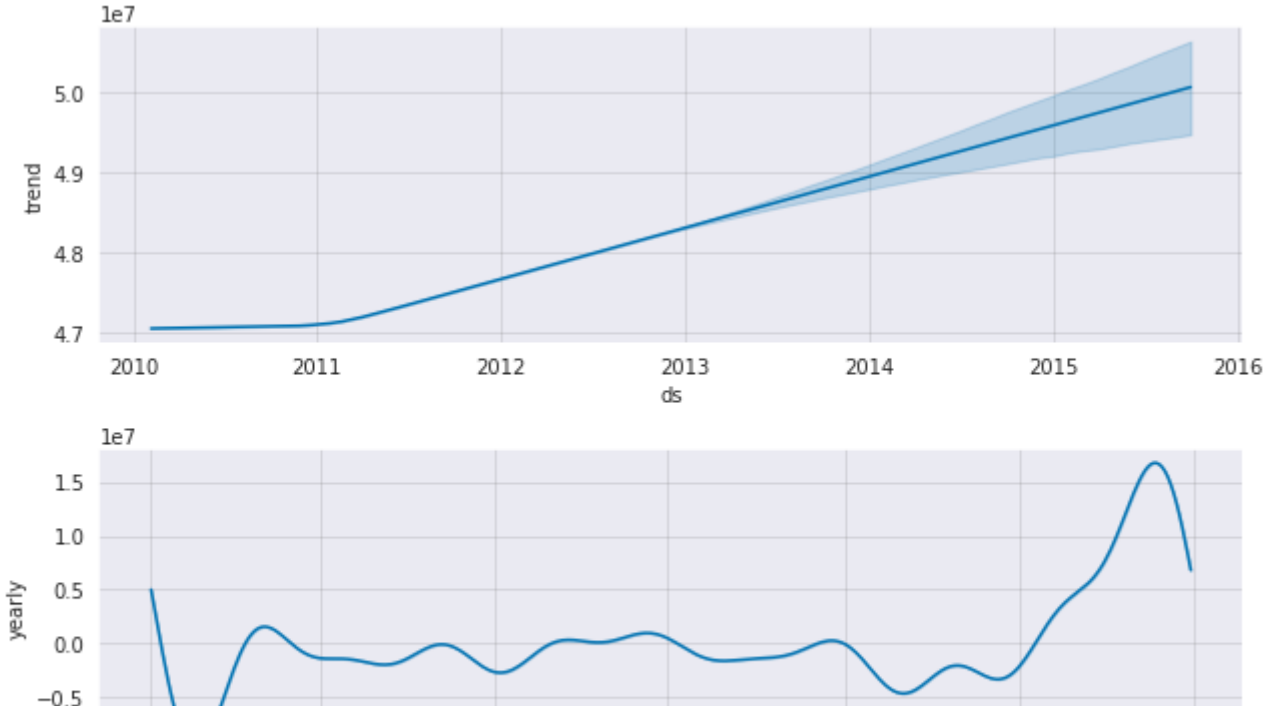
```
sales_forecast
```

	ds	trend	yhat_lower	yhat_upper	trend_lower	trend_upper	add
0	2010-02-05	4.704915e+07	4.060451e+07	5.423157e+07	4.704915e+07	4.704915e+07	
1	2010-02-12	4.704991e+07	4.183270e+07	5.565546e+07	4.704991e+07	4.704991e+07	
2	2010-02-19	4.705066e+07	4.049866e+07	5.383766e+07	4.705066e+07	4.705066e+07	
3	2010-02-26	4.705142e+07	3.893320e+07	5.265377e+07	4.705142e+07	4.705142e+07	-
4	2010-03-05	4.705218e+07	3.843046e+07	5.268059e+07	4.705218e+07	4.705218e+07	-
...	...	...	...	...	...	...	
174	2015-06-01	4.984219e+07	4.312558e+07	5.675985e+07	4.935584e+07	5.031646e+07	
175	2015-	4.989454e+07	4.391417e+07	5.719256e+07	4.938220e+07	5.039685e+07	

```
plt.figure(figsize=(20, 8))
sales_model.plot(sales_forecast, xlabel = 'Date', ylabel = 'Sales')
plt.title('Weekly Sales')
```




```
sales_model.plot_components(sales_forecast);
```



There is an increasing trend for weekly sales. Regarding day of the year, the worst sales are in January, the best sales are in December.

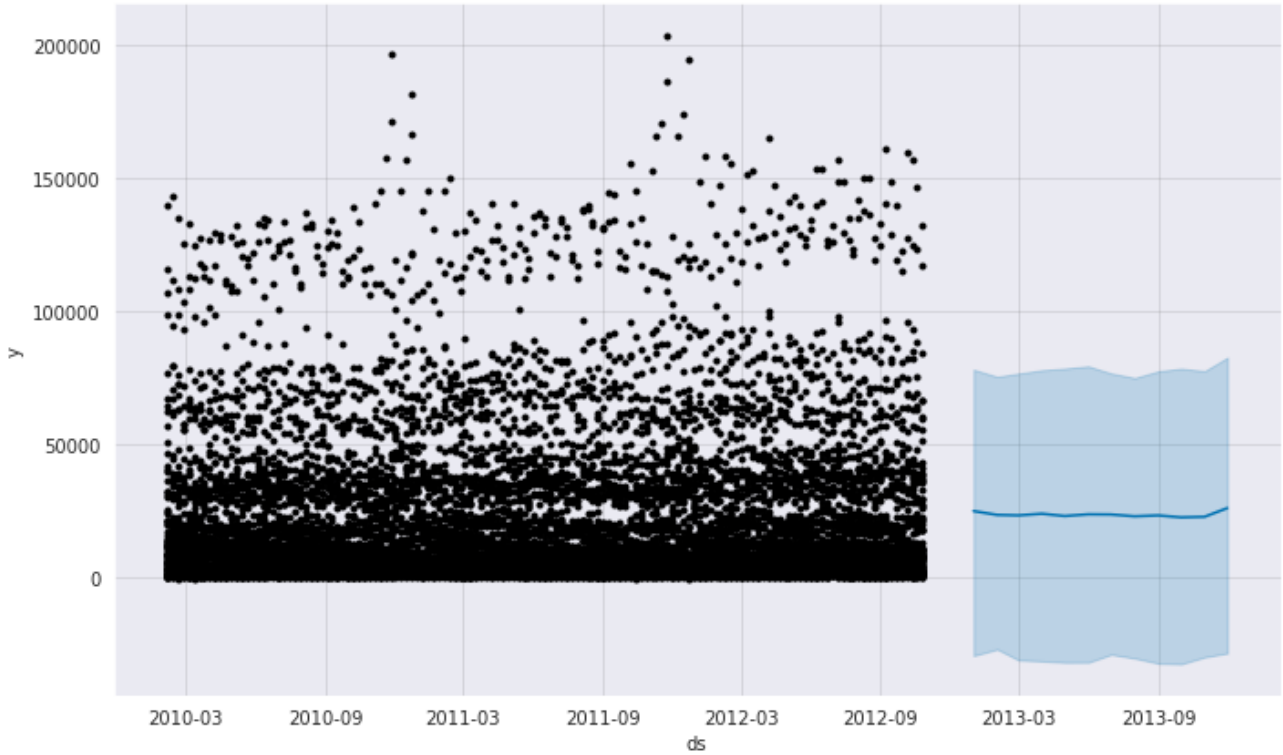
```
# train = pd.read_csv(io.BytesIO(uploaded['train.csv']))
# train = train[['Date','Store', 'Weekly_Sales']].rename(columns={'Date': 'ds','Sto

# store2df = {}
# forecast = pd.DataFrame()
# for i in range(1,46):
#   store2df[i] = train[train['Store']== i]
#   # store2df[i]['ds'] = train.index
#   # store2df = store2df[['ds', 'Weekly_Sales']].rename(columns={'ds': 'ds', 'Week
#   sales_model = Prophet(seasonality_mode='additive',interval_width=0.95,weekly_se
#   sales_model.fit(store2df[i])
#   sales_forecast = sales_model.make_future_dataframe(periods=45, freq='W')
#   sales_forecast = sales_model.predict(sales_forecast)[['ds', 'yhat']]
#   forecast = forecast.append(sales_forecast)
#   # print(sales_forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail())

# forecast.to_csv('forecast_weekly.csv')


# # forecast_by_store = pd.DataFrame(forecast, columns = ['ds', 'Weekly_Sales'])
```

```
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
```

```
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
INFO:fbprophet:Disabling daily seasonality. Run prophet with daily_seasonalit
```

```
# from pandas import DataFrame
# from pandas import to_datetime
# from matplotlib import pyplot
# future = list()
# for i in range(1, 13):
#   date = '2013-%02d' % i
#   future.append([date])
# future = DataFrame(future)
# future.columns = ['ds']
# future['ds']= to_datetime(future['ds'])
# # # use the model to make a forecast
# forecast = sales_model.predict(future)
# forecast
# # # summarize the forecast
# print(forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].head())
# # # plot forecast
# sales_model.plot(forecast)
# pyplot.show()
```

```
          ds          yhat    yhat_lower    yhat_upper
0 2013-01-01  24732.652116 -29579.282059  77944.143768
1 2013-02-01  23298.984960 -27181.340332  75278.529401
2 2013-03-01  23117.224137 -31219.440135  76473.399739
3 2013-04-01  23731.270843 -31632.707043  77752.327319
4 2013-05-01  22869.803952 -32105.936826  78378.271522
```



✓ 0s completed at 9:58 PM